

# Fused MoE Inference on Blackwell: A Workload-Adaptive Kernel System for DeepSeek-V3

Jiayao Zhang, Jiaoliang Yu  
MLSys 2026 FlashInfer-Bench Challenge — Track A

## Abstract

We present an agent-assisted, workload-adaptive implementation of the DeepSeek-V3 MoE kernel targeting the NVIDIA B200 (Blackwell, sm\_100a) GPU. Our fused Triton pipeline exploits FP8 native tensor core dot products for  $2\times$  throughput in GEMM1, a non-atomic two-pass GEMM2-then-reduce architecture, FP16 intermediate buffers with scale-and-cast compensation, and multi-level bucket specialization across BLOCK\_M tiers. For the largest traces, a hybrid Triton/CuTe path uses Triton for sparse MoE metadata and epilogues while delegating the grouped GEMM core to CuTe. On 19 real-trace workloads ( $T=1$  to  $T=14,107$ ), the system achieves 19/19 correctness with a peak speedup of  $\sim 56\times$ , large- $T$  speedups of  $\sim 13\text{--}15\times$ , and a mean of  $\sim 54\times$  on reference Modal B200 sessions, as measured by CUPTI GPU-only kernel timing.

## 1 Introduction

Mixture-of-Experts (MoE) architectures enable massive parameter counts with sub-linear compute growth. DeepSeek-V3 [1] uses 256 routed experts (32 local per device), top-8 routing, hidden dimension 7168, and intermediate dimension 2048—yielding two FP8 block-scaled GEMMs per forward pass.

On NVIDIA B200, naïve per-expert execution wastes hardware: launch overhead dominates small traces, while large traces suffer from atomic scatter in reduction.

This work was developed for Track A of the MLSys 2026 FlashInfer-Bench Challenge [4], where scoring is the sum of CUPTI GPU kernel times across 19 workloads (tolerance atol=1.0, rtol=0.3, ratio=0.9). CUPTI excludes CPU overhead, so optimization is driven by GPU kernel efficiency. Our contributions:

1. A fused Triton [2] pipeline eliminating per-expert launches;
2. Native FP8 tensor core dot in GEMM1, delivering  $2\times$  throughput over TF32;

3. Non-atomic GEMM2 with token-centric reduce, yielding +46% on medium- $T$ ;
4. FP16 intermediate buffers with  $\times 0.125/\times 8.0$  compensation ( $-50\%$  bandwidth);
5. Workload-aware dispatch, including singleton, tight-padding, isolated transition, and CuTe-assisted large- $T$  buckets.

## 2 System Architecture

### 2.1 Pipeline Overview

Figure 1 shows the generic five-stage pipeline: **(1) Routing:** Triton DeepSeek-V3 no-aux routing (sigmoid, group top-4, global top-8, normalization) with one program per token. **(2) Sorting:** Tile-level parallel histogram with atomic offsets and padded row construction. **(3) GEMM1+SwiGLU:** Fused  $W_1/W_3$  projections in a shared K-loop with `tl.dot(fp8, fp8)`. **(4) GEMM2:** Non-atomic down-projection writing to the `expert_out` buffer. **(5) Token Reduce:** Each output token reads  $K=8$  expert contributions, accumulates in FP32, writes bf16.

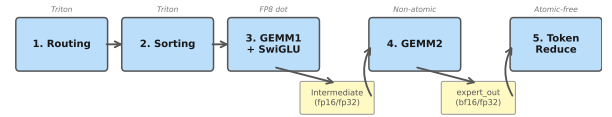


Figure 1: Generic MoE pipeline. Singleton and large- $T$  workloads reuse the same stages with different kernels.

### 2.2 Dispatch and Specialization

Workloads span four orders of magnitude, so a single tile shape is suboptimal. We therefore organize execution around a default BLOCK\_M tiering rule plus a small number of explicit trace-specific overrides (Table 1). The main overrides are: singleton  $T=1$ , tight BLOCK\_M=8 for fragmented  $T=53\text{--}59$ , isolated  $T=901$ , and a hybrid Triton/CuTe path for the largest traces. For  $T\geq 4096$ ,

the dispatcher switches from static overlunch to exact block-count launch using `total_blocks` produced by sorting, avoiding  $\sim 30\%$  idle blocks.

Table 1: Workload-adaptive dispatch.

| Regime                 | BM Path                           |
|------------------------|-----------------------------------|
| $2 \leq T \leq 128$    | 16 Generic Triton                 |
| $129 \leq T \leq 512$  | 32 Generic Triton                 |
| $513 \leq T \leq 2047$ | 64 Generic Triton                 |
| $T \geq 2048$          | 128 Generic Triton (exact launch) |
| $T=1$                  | 16 Singleton Triton               |
| $T=53-59$              | 8 Tight-padding Triton            |
| $T=901$                | 64 Isolated Triton                |
| $T=11948, 14107$       | 128 Triton/CuTe hybrid            |

### 3 Key Optimizations

#### 3.1 FP8 Native Tensor Core Dot

Blackwell’s `tcgen05.mma` supports native  $\text{FP8} \times \text{FP8}$  dot with  $\text{FP32}$  accumulation. We apply block-scale dequantization *after* the dot:

```
partial = tl.dot(a, b) # fp8 x fp8
acc += partial * a_scale * b_scale
```

This yields  $\sim 2\times$  throughput vs.  $\text{TF32}$ , which requires  $\text{FP8} \rightarrow \text{FP32}$  dequantization before the dot.  $\text{GEMM1}$  stores activations and weights in `float8_e4m3fn` with per-128 scaling and fuses  $W_1/W_3$  with inline `SwiGLU`.

#### 3.2 Non-Atomic GEMM2 + Token-Centric Reduce

Naïve `tl.atomic_add` causes  $6\times$  bandwidth amplification (12 B/elem  $\text{FP32}$  RMW vs. 2 B  $\text{bf16}$  store). We split  $\text{GEMM2}$  and reduction:  $\text{GEMM2}$  writes non-atomic `expert_out` ( $\text{bf16}$  where enabled), then a token-centric reduce loads 8 contributions per token. For large  $T$ , 2D tiling cuts the reduce grid  $>10\times$ . A/B testing confirmed this as one of the largest single gains, especially on medium- $T$  workloads where atomic contention previously dominated.

#### 3.3 FP16 Intermediate with Scale Compensation

The Intermediate buffer `[num_padded, 2048]` dominates HBM traffic. We scale by  $\times 0.125$  in  $\text{GEMM1}$ , store  $\text{FP16}$ , then compensate by  $\times 8.0$  in  $\text{GEMM2}$ . For  $T < 32$ , a `constexpr` branch falls back to  $\text{FP32}$  to avoid `SwiGLU` outlier overflow. A/B: mean  $+4.5\%$  (13/19 improved).

#### 3.4 Workload-Adaptive Bucket Specialization

Column-major dispatch (`GROUP_M=32/64`) improves  $\text{L2}$  reuse by mapping consecutive blocks to different tokens of the same expert. The  $T=1$  path is a dedicated decode implementation: it bypasses generic histogram/offset metadata and uses two fused kernels ( $\text{GEMM1} + \text{SwiGLU}$  and  $\text{GEMM2} + \text{reduce}$ ) around a fixed small workspace. For fragmented  $T=53-59$ , `BLOCK_M=8` reduces padding and invalid-row compute. For  $T=901$ , an isolated path targets the transition regime.

#### 3.5 Routing, Sorting, and Autotuning Support

Several supporting optimizations are smaller individually but important to the final system. Routing uses a Triton kernel implementing group-filter then global top-K selection with pre-allocated workspaces; sorting uses tile-parallel histogram/offset construction and, for large traces, reuses the histogram structure in dispatch. Autotuning remains workload-specific:  $\text{GEMM1}/\text{GEMM2}$  tile shapes, `BLOCK_K`, and stage depth are selected per regime rather than by a single global configuration.

#### 3.6 CuTe-Assisted Large-T GEMM

For the largest traces, bottlenecks shift from launch/padding overhead to sustained grouped  $\text{GEMM}$  throughput. We therefore use a hybrid path for  $T=11,948$  and  $T=14,107$ : Triton handles sparse metadata and epilogues, while CuTe executes the grouped  $\text{GEMM}$  core. We do not use CuTe for small/mid- $T$ , where metadata and setup dominate.

#### 3.7 bf16 Expert Output Buffer

The `expert_out` buffer `[MAX_PADDED, 7168]` is stored in  $\text{bf16}$  ( $T \geq 32$ ), saving 50% write bandwidth.  $\text{bf16}$ ’s  $3.4 \times 10^{38}$  range avoids the  $\text{fp16}$  overflow that caused `inf` on 3/19 workloads. A/B:  $+0.4\%$  mean,  $+2.5\%$  at  $T=14,107$ .

#### 3.8 Compiler Hints and Cache Policy

Two compiler hints yielded a combined  $+4.9\%$  mean speedup: **(1) Disable LLVM LSR:** Setting `DISABLE_LLVM_OPT="disable-lsr"` avoids extra register pressure and spills in `SwiGLU` addressing. **(2) Asymmetric L2 eviction:** Expert weights use `evict_last`; token activations use `evict_first`.

Table 2: Failed optimization attempts.

| Approach           | Result  | Root Cause                                     |
|--------------------|---------|------------------------------------------------|
| FP8 Intermediate   | 0/19    | 3-bit mantissa; SwiGLU exceeds fp8 range       |
| bf16 Intermediate  | 8/19    | Low mantissa precision; ptxas register failure |
| Persistent GEMM2   | −1.0%   | K=2048 too short for weight reuse              |
| TMA Descriptor     | −4%     | Setup cost not amortized over 16 K-tiles       |
| Warp Specialize    | Crash   | TTGIR lowering failure; slow fallback          |
| Fused GEMM2+Reduce | −4.5%   | fp32 atomic RMW costs $6 \times$ bf16 store BW |
| CuTe for mid- $T$  | No gain | Metadata and host sync not amortized           |
| GEMM2 FP8 Dot      | 5/19    | fp16→fp8 cast remains too lossy                |

### 3.9 2D Tiled Token Reduce

The original 1D reduce launched one program per (token, N-block) pair, producing a large grid at  $T=14,107$ . Switching to 2D [BLOCK\_T, BLOCK\_N] tiling amortizes the per-token loop overhead across BLOCK\_T rows, cutting the grid  $>10\times$  and improving occupancy. A/B: +2.5–2.9% on the largest traces.

### 3.10 Deep Software Pipelining for GEMM2

GEMM2’s FP16 A-side incurs high HBM latency. Extending autotuning to `num_stages=6/7/8` overlaps compute with reads and yields +2.1% mean (+10.1% peak) on medium- $T$  workloads.

## 4 Negative Results

Table 2 summarizes the main rejected experiments.

The failures fall into two groups: lower-precision activation buffers break correctness (Figure 2), while persistence, TMA, fusion, and mid- $T$  CuTe do not amortize setup, metadata, or atomic costs. We retain only changes that pass correctness and exceed the same-session noise floor.

|                                 | FP8<br>(3-bit)            | bf16<br>(7-bit)   | FP16<br>(10-bit) | FP32<br>(23-bit)  |
|---------------------------------|---------------------------|-------------------|------------------|-------------------|
| Intermediate<br>(GEMM1 → GEMM2) | 0/19<br>abs>10K ptxas fai | 8/19<br>ptxas fai | 19/19<br>+4.5%   | 19/19<br>baseline |
| expert_out<br>(GEMM2 output)    | N/A                       | 19/19<br>+0.4%    | 3/19<br>overflow | 19/19<br>baseline |

19/19 PASS    Partial pass    FAIL (0/19)

Figure 2: Precision compatibility matrix. Intermediate requires  $\geq$ FP16; expert\_out tolerates bf16.

## 5 Evaluation

**Setup.** We evaluate with the official FlashInfer-Bench CLI on NVIDIA B200 (sm\_100a). Scoring uses CUPTI GPU kernel time over 19 real-trace workloads; CPU overhead is excluded. All claims use same-session A/B testing with a  $\pm 2\%$  mean noise floor.

### 5.1 Results

Figure 3 and Table 3 summarize representative performance by workload regime. **Correctness:** 19/19. **Best observed speedup:**  $\sim 56\times$ . **Mean speedup:**  $\sim 54\times$ . Small and fragmented traces benefit most because the baseline pays repeated per-expert launch overhead while our fused path amortizes routing, sorting, and GEMM dispatch.  $T=1$  is best treated as a singleton specialization rather than part of the grouped-kernel trendline. The two largest traces sustain  $\sim 13\text{--}15\times$  speedup in reference Modal sessions.

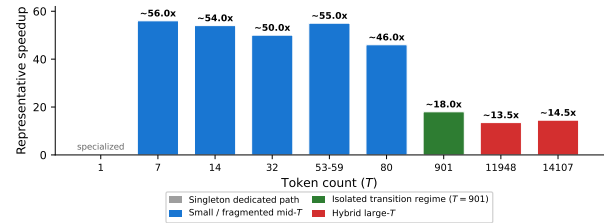


Figure 3: Representative speedup by workload regime.

Table 3: Representative benchmark regimes.

| $T$         | Traces            | Speedup                    | Bottleneck |
|-------------|-------------------|----------------------------|------------|
| 1           | singleton decode  | dedicated                  | GEMM2      |
| 7–80        | small/mid traces  | $\sim 46\text{--}56\times$ | GEMM1      |
| 901         | transition regime | $\sim 18\times$            | GEMM2      |
| 11948–14107 | large- $T$        | $\sim 13\text{--}15\times$ | GEMM2      |

### 5.2 Profiling Analysis

NCU profiling reveals three bottleneck regimes (Figures 4–5): **Small/medium  $T$  (7–80):** GEMM1 dominates because expert weights are repeatedly loaded for very few routed rows, with substantial padding waste.  **$T=901$ :** the workload sits in a GEMM1/GEMM2 transition regime, motivating an isolated path. **Large hybrid  $T$  (11,948–14,107):** GEMM1 still takes the larger time share ( $2\times$  the FLOPs from fused  $W_1/W_3$ ), but GEMM2 is the efficiency bottleneck—its FP16 A-side prevents FP8 tensor core use, capping throughput well below peak. CuTe improves the grouped-GEMM core. Token reduce remains lightweight across regimes.

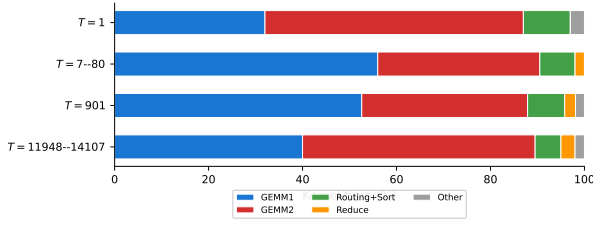


Figure 4: Representative kernel-time breakdown by workload regime.

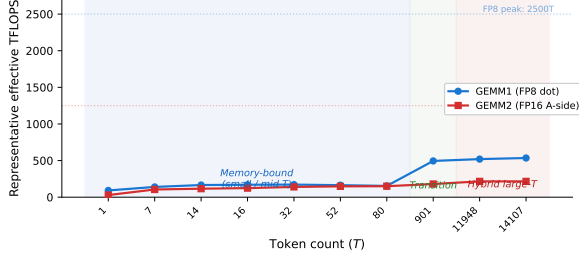


Figure 5: Representative effective TFLOPS vs. token count.

### 5.3 Optimization Impact

Figure 6 shows cumulative impact over 23 rounds of optimization. The largest jumps come from eliminating per-expert launches, building the FP8-dot/non-atomic GEMM2 backbone, and then adding workload-aware specialization. Later gains are smaller but cumulative: compiler hints, deeper GEMM2 pipelining, bf16 expert\_out, exact large- $T$  dispatch, isolated runtimes, and tight mid- $T$  BLOCK\_M tuning.

### 5.4 Measurement Noise

Modal B200 shared instances exhibit  $\pm 2\%$  mean noise and  $\pm 15\%$  per-workload noise for small  $T$ , but  $< 1\%$  for large  $T$ . Cross-session drift reaches  $\pm 20-30\%$ ; all claims use same-session A/B testing with a 2% mean threshold. The contest tolerance (atol=1.0, rtol=0.3, 90%) is  $\sim 100\times$  wider than our internal checks, enabling bf16 expert output.

## 6 Tools & Languages

We implement in **Python**, using **Triton** [2] for routing/sorting/GEMM/reduce kernels and **CuTe DSL** for the two largest grouped-GEMM workloads. We evaluated raw CUDA, Tilelang, and CuTile, but retained Triton/CuTe because Triton’s FP8 intrinsics and autotuner covered 17/19 workloads efficiently and CuTe cleanly handled the remaining two.

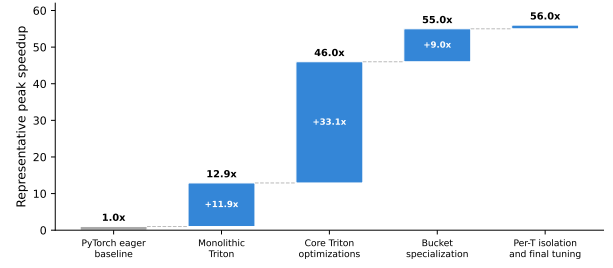


Figure 6: Representative cumulative speedup across major optimization phases.

## 7 Human vs. Agent Contributions

Human contributors led the profiling-driven strategy: they identified workload-adaptive dispatch as the organizing principle, decided when pure Triton had reached diminishing returns, proposed introducing CuTe DSL for the largest grouped-GEMM traces, and selected which experiments to keep or revert. They also interpreted benchmark noise, made final correctness/risk decisions, and maintained the contest-facing solution. The agent assisted by inspecting profiler traces, expanding human hypotheses into bounded Triton/CuTe experiments, comparing A/B results, and summarizing failed directions. All final optimizations were human-reviewed against correctness, benchmark stability, and simplicity before inclusion.

## 8 Conclusion

We presented a workload-adaptive MoE kernel system achieving  $\sim 56\times$  peak and  $\sim 54\times$  mean speedup on B200 through 15 rounds of profiling-driven optimization, combining FP8 dot, non-atomic GEMM2, FP16 intermediate, workload-aware dispatch, and CuTe-assisted large- $T$  GEMM. Eight rejected approaches fall into precision failures and amortization failures where  $K=2048$  does not justify setup cost.

## References

- [1] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv:2412.19437*, 2024.
- [2] P. Tillet, H. T. Kung, D. Cox. Triton: An intermediate language and compiler for tiled neural network computations. *MLSys*, 2019.
- [3] B. Darvish Rouhani et al. Microscaling data formats for deep learning. *arXiv:2310.10537*, 2023.
- [4] Z. Ye et al. FlashInfer: Efficient and customizable attention engine for LLM inference serving. *arXiv:2501.01005*, 2025.